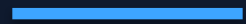


STATE-SPACE & RK4

Bringing Newton's 2nd law to life.



Numerical integration meets interactive matplotlib —
slide a slider, watch the physics respond.

Where we left off — Day 5.

Day 5 — Exceptions

- try / except / raise / finally — control flow for failure paths.
- Custom exception classes (NegativeDimensionError, ...).
- EAFP over LBYL — ask forgiveness, not permission.
- Fixed vector3d.py — normalize() now raises on zero magnitude.
- JSON scene loader with FileNotFoundError + JSONDecodeError handling.

Day 6 — what changes today

- Move from defensive code → numerical simulation.
- ODE → state-space form (the standard control-systems trick).
- RK4 integrator — port the one from EGR105 ISS Deorbit.
- matplotlib.widgets.Slider + RadioButtons — live parameter tweaking.
- JSON again — but now for config save/load (full circle from Day 5).



From correctness → behavior.

Day 5 made the code survive bad input. Day 6 makes the code show you something.

ODE → state-space form.

Newton's 2nd law (1 DOF):
 $m \cdot (d^2x/dt^2) + c \cdot (dx/dt) + k \cdot x = F(t)$

*# Let: $x1 = x$ (position)
 # $x2 = dx/dt$ (velocity)*

*# Then the 2nd-order ODE becomes
 # two coupled 1st-order ODEs:*

**$dx1/dt = x2$
 $dx2/dt = (F(t) - c \cdot x2 - k \cdot x1) / m$**

*# In vector form: $dx/dt = f(t, x)$
 # state vector $x = [x1, x2]^T$*

Why this form?

- Every integrator (Euler, RK4, scipy) expects 1st-order ODEs.
- A 2nd-order ODE → split into n coupled 1st-order ones.
- Lets you treat position and velocity as one state vector X .
- Extends cleanly to multi-DOF systems later (n masses → 2n states).



State-space is the lingua franca of dynamics.

If you can write it as $dx/dt = f(t, x)$, any solver in any language can integrate it.

RK4 — four slopes, one step.

```
def rk4_step(f, t, x, dt):
    """Advance one timestep of  $dx/dt = f(t, x)$ ."""
    k1 = f(t, x)
    k2 = f(t + dt/2, x + (dt/2)*k1)
    k3 = f(t + dt/2, x + (dt/2)*k2)
    k4 = f(t + dt, x + dt * k3)

    return x + (dt/6) * (k1 + 2*k2 + 2*k3 + k4)

# x is a numpy array — k1..k4 are arrays too
# weighted slope average → 4th-order accurate (error ~  $dt^5$ )
```

Why RK4 over Euler?

- Euler uses one slope — drifts fast for oscillators.
- RK4 samples slope at 4 points, weighted (1, 2, 2, 1)/6.
- Error per step $\sim O(dt^5)$. For $dt=0.01$ that's tiny.
- Same skeleton you wrote for ISS Deorbit — port it.
- If energy drifts in the phase plot, your dt is too big.

 **Same integrator, different $f(t, x)$.**

Spring-mass-damper, orbiting satellite, RC circuit — RK4 doesn't care. Write f , plug it in.

matplotlib.widgets.Slider

```
from matplotlib.widgets import Slider

# Reserve space on the figure for sliders below the plot
fig.subplots_adjust(bottom=0.30)

# Each slider needs its own axes (a thin rectangle)
ax_m = fig.add_axes([0.15, 0.18, 0.65, 0.03])
ax_k = fig.add_axes([0.15, 0.13, 0.65, 0.03])
ax_c = fig.add_axes([0.15, 0.08, 0.65, 0.03])

s_m = Slider(ax_m, "mass m", 0.1, 10.0, valinit=1.0)
s_k = Slider(ax_k, "spring k", 0.1, 50.0, valinit=10.0)
s_c = Slider(ax_c, "damping c", 0.0, 10.0, valinit=0.5)

s_m.on_changed(update_sim) # callback re-runs the integration
```

How it wires up

- axes = the physical rectangle on the figure (x, y, w, h in fig fraction).
- Slider needs its own axes — separate from the plot.
- on_changed(fn) — fn fires on every drag.
- Inside fn: re-integrate, update Line2D data via .set_data().
- Call fig.canvas.draw_idle() to re-render.

Phase-space + damping regime.

```
# Two subplots: time series + phase space
fig, (ax_t, ax_ph) = plt.subplots(1, 2)

ax_t.set_xlabel("t (s)"); ax_t.set_ylabel("x (m)")
ax_ph.set_xlabel("x (m)"); ax_ph.set_ylabel("v (m/s)")

# Damping ratio  $\zeta = c / (2\sqrt{m\cdot k})$ 
def regime(m, c, k):
    zeta = c / (2 * (m*k)**0.5)
    if zeta < 1: return "underdamped"
    if zeta == 1: return "critically damped"
    return "overdamped"
```

Reading the phase plot

- Spiral inward → underdamped ($\zeta < 1$).
- Straight line into origin → critically damped ($\zeta = 1$).
- Lazy curve, no overshoot → overdamped ($\zeta > 1$).
- $\zeta = c / (2\sqrt{mk})$ — float comparison to 1 is sketchy; treat $|\zeta - 1| < 1e-3$ as critical.
- Update the label text each callback:
ax.set_title(regime(m,c,k)).

Forcing options — RadioButtons.

```
from matplotlib.widgets import RadioButtons
```

```
def free(t):    return 0.0
```

```
def step(t):    return 1.0 if t > 1.0 else 0.0
```

```
def sinusoid(t):
```

```
    return A * np.sin(omega_f * t)
```

```
FORCES = {"free": free, "step": step, "sin": sinusoid}
```

```
ax_radio = fig.add_axes([0.83, 0.05, 0.12, 0.15])
```

```
radio = RadioButtons(ax_radio, ("free", "step", "sin"))
```

```
radio.on_clicked(lambda label: set_force(FORCES[label]))
```

Threading forcing through

- Forcing $F(t)$ lives inside $f(t, x)$ — your state-derivative function.
- Closure trick: capture `current_force` in the outer scope, swap it.
- Or pass `force_fn` explicitly to `rk4_step` via `functools.partial`.
- Step forcing — watch the system settle to F/k offset.
- Sinusoidal at $\omega_f \approx \omega_n \rightarrow$ resonance. Amplitude blows up if c is small.

JSON config — save & load.

```
import json, pathlib

def save_config(path, m, k, c, x0, v0, force):
    cfg = {"m": m, "k": k, "c": c,
          "x0": x0, "v0": v0, "force": force}
    pathlib.Path(path).write_text(json.dumps(cfg, indent=2))

def load_config(path):
    try:
        return json.loads(pathlib.Path(path).read_text())
    except (FileNotFoundError, json.JSONDecodeError) as e:
        print(f"config load failed: {e}")
        return DEFAULT_CONFIG
```

Why bother

- Reproduce a tuning hours later without re-dragging sliders.
- Share specific scenarios (resonance demo, hard step input).
- Same exception pattern as Day 5's scene loader — reuse the muscle.
- Wire two Buttons: 'save' → write file, 'load' → set slider values via `s_m.set_val(...)`.

 **Sliders for exploration, JSON for reproduction.**

If you find something interesting, save it. Future-you won't remember the exact values.

Spring-Mass-Damper sandbox.

Build `smd_matplotlib.py` — a live, sliderable, configurable mass-spring-damper rig.

STRUCTURE

Modules & files

- `smd_model.py` — pure physics, $f(t, x)$, `rk4_step`, `regime()`.
- `smd_view.py` — matplotlib figure, sliders, radio buttons, JSON save/load.
- `smd_matplotlib.py` — entry point. Imports both.
- `configs/` folder — sample JSON presets.

PHYSICS

Model layer

- State vector $X = [\text{pos}, \text{vel}]$; `numpy.array`, length 2.
- $f(t, X, m, c, k, F_{\text{fn}})$ returns $[v, (F - cv - kx)/m]$.
- `rk4_step(f, t, X, dt)` — port from EGR105 ISS Deorbit.
- `regime(m, c, k) → 'under' / 'critical' / 'over'` via $\zeta = c/(2\sqrt{mk})$.

INTERACTION

View layer

- Two subplots: x vs t (left), phase plot v vs x (right).
- Sliders: m, k, c, x_0, v_0 — plus ω_f and amplitude for sinusoid.
- RadioButtons: free / step / sinusoidal.
- Live damping-regime label (set as title or text annotation).
- Two buttons: save config, load config. Reuse Day 5's try/except.

Hard rule: model/view stay separate. Tomorrow's pygame port should reuse `smd_model.py` untouched.

Things that will bite.

⚠ **dt too big = energy gain.**

If your phase plot spirals OUT instead of in (with $c > 0$), drop dt. RK4 is stable but not magic. Start with $dt = 0.01$ for $k/m \sim 10$; smaller if stiff.

⚠ **Re-create lines, don't append.**

On each slider callback, recompute the full trajectory and call `line.set_data(t, x)`. Don't `ax.plot()` again — you'll stack lines and the figure leaks memory.

💡 **Slider axes $\neq$ plot axes.**

Sliders need their own `fig.add_axes([...])` rectangles. Reserve space with `subplots_adjust(bottom=0.3)` or they'll overlap the plot.

💡 **Closure for forcing.**

RadioButtons callback gets a string label — map it to a function. Store current `force_fn` in a one-element list so the callback can mutate it (Python closures capture by name).

Today → tomorrow → next week.

TOMORROW · DAY 07

pygame port

Same `smd_model.py` — different view. pygame loop, keyboard controls (+/- on m/k/c, [] on ω_f , space = pause). Render the mass as a Box from Day 3's shapes module. The model/view separation pays off.

NEXT WEEK · pygame

Game loop fluency

Week 2 is Pong. Today's RK4 + Day 3's shapes + Day 4's classmethods = the physics layer you'll need. The 60 Hz game loop is just integration with a fixed dt.

LATER · CRUX HUD

Audio-reactive orb

Wk 10's voicebox orb is a forced oscillator — audio amplitude drives $F(t)$, the geometry deforms. Today's $f(t, x)$ skeleton is exactly that, just visualized differently.

The integrator you wrote today will keep showing up. Treat it as core infrastructure.